ELO328 - Procesamiento Digital de Imágenes

Clase 18: Descriptores locales

Prof. Nicolás Torres
nicolas.torresr@usm.cl
Ingeniería Civil Telemática
Departamento de Electrónica
Universidad Técnica Federico Santa María

Esquinas

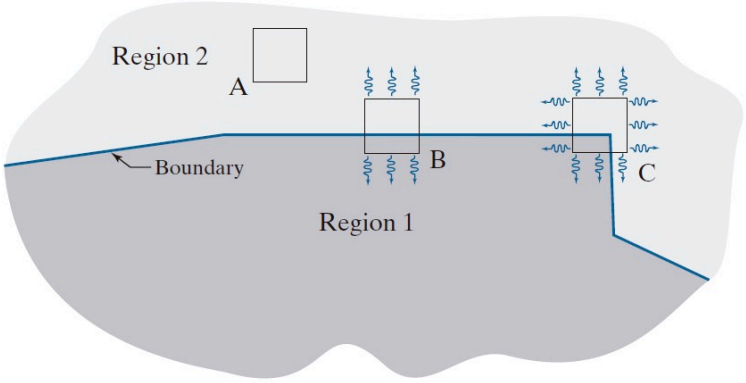
- Intuitivamente, podemos pensar en una esquina como un cambio brusco de dirección en una curva.
- Las esquinas son características muy útiles porque son distintivas y relativamente invariables si miramos desde cualquier punto de vista.
- Debido a estas características, las esquinas se utilizan habitualmente para hacer *matching* de características en aplicaciones como: *tracking* de vehículos por ejemplo.

Detector de Esquinas de Harris-Stephens

- La idea es detectar esquinas desplazando una pequeña ventana sobre la imagen.
- La ventana está diseñada para detectar los cambios de intensidad y cubre 3 escenarios posibles:

1. Áreas de cero (o pequeños) cambios de intensidad en todas las direcciones. [A]
2. Áreas de cambios en una dirección pero sin cambios en la dirección ortogonal. [B]
3. Áreas de cambios significativos en todas las direcciones, una condición que ocurre cuando la ventana contiene una esquina (o puntos aislados). [C]

- El detector de esquinas de Harris es una formulación matemática que intenta diferenciar entre estas tres condiciones.



El detector de esquinas de Harris opera en tres tipos de regiones: A (uniforme), B (borde) y C (esquina).

Formalización

Sea f una imagen, se define una matriz de Harris $\mathbf{M}$ tal como:

$$\mathbf{M} = \sum_u \sum_v w(u,v)\mathbf{A}$$

con

$$\mathbf{A} = \begin{bmatrix} f_x^2 & f_x f_y \\ f_x f_y & f_y^2 \end{bmatrix}$$

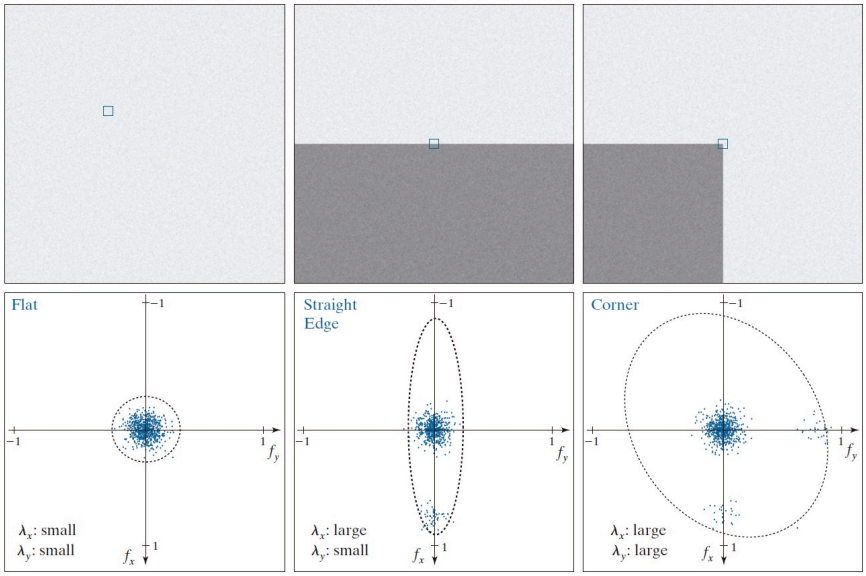
La función de pesos w es la ventana que puede ser rectangular, aunque los autores proponen una Gaussiana:

$$w(u,v) = e^{-\frac{(u^2+v^2)}{2\sigma^2}}$$

¿Cómo sabe la ventana si está en una región uniforme, en un borde, o en una esquina?

Los vectores propios de una matriz simétrica real (como $\mathbf{M}$) siempre apuntan en la dirección de máxima dispersión de los datos, y los valores propios correspondientes son proporcionales a la cantidad de dispersión de los datos en la dirección de los vectores propios.

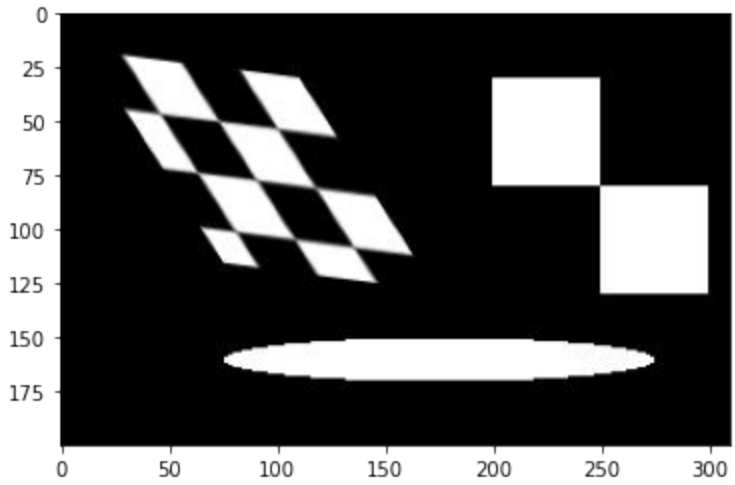
- Los valores propios λ_1 y λ_2 de $\mathbf{M}$, son proporcionales a las curvaturas principales de la imagen f.
- Basándose en ellos, la región puede clasificarse como:
  - Uniforme (Flat): λ_1 y λ_2 pequeños.
  - Borde (Edge): Uno pequeño y otro grande.
  - Esquina (Corner): λ_1 y λ_2 grandes.



In [1]:
```
from matplotlib import pyplot as plt
```

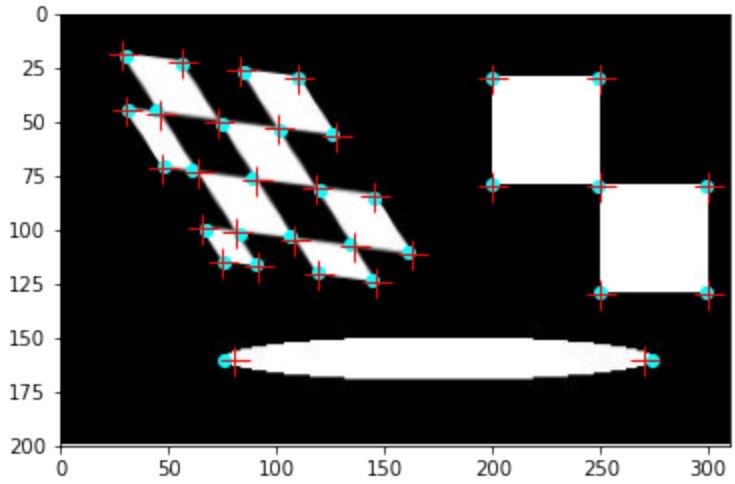
In [2]:
```
from skimage import io
from skimage import filters
from skimage.color import rgb2gray
from skimage.feature import corner_harris, corner_subpix, corner_peaks
```

In [3]:
```
image = io.imread("img/corners.jpg")
image = rgb2gray(image)
plt.imshow(image, cmap='gray')
plt.show()
```



In [4]:
```
coords = corner_peaks(corner_harris(image), min_distance=5, threshold_rel=0.02)
coords_subpix = corner_subpix(image, coords, window_size=13)
```

In [5]:
```
fig, ax = plt.subplots()
ax.imshow(image, cmap=plt.cm.gray)
ax.plot(coords[:, 1], coords[:, 0], color='cyan', marker='o', linestyle='None',
ax.plot(coords_subpix[:, 1], coords_subpix[:, 0], '+r', markersize=15)
ax.axis((0, 310, 200, 0))
plt.show()
```



SIFT

- Scale-Invariant Feature Transform (SIFT) es un algoritmo para detectar características en una imagen.
- Es uno de los más populares, ya que funciona incluso cuando las imágenes tienen diferente escala y orientación.
- Las características que detecta SIFT (*keypoints*) son invariantes a la escala, rotación, e incluso a la iluminación.

Scale Space

- La primera etapa de SIFT es encontrar las partes de la imagen que son invariantes al cambio de escala.
- Esto se logra mediante la búsqueda de características estables en todas las escalas posibles, utilizando una función conocida como *Scale Space*, que es una representación en diferentes escalas de la imagen.
- La idea es analizar la pérdida de detalle que ocurre a medida que disminuye la escala de la imagen y cuales son los detalles que se mantienen.
- SIFT utiliza kernel gaussiano para realizar un suavizado y el parámetro que controla la escala es la desviación estándar.

$$G(x, y, \sigma) = \frac{1}{2\pi\sigma^2} e^{-(x^2+y^2)/2\sigma^2}$$

In [6]:

```
house = io.imread("img/house.jpg")
plt.figure(figsize = (7,7))
for i in range(9):
    plt.subplot(3, 3, i + 1)
    plt.axis('off')
    plt.imshow(filters.gaussian(rgb2gray(house), sigma=i*1.5), cmap='gray')
plt.subplots_adjust(wspace=0, hspace=0)
```



Puntos de interés (*Keypoints*)

Las ubicaciones de los puntos de interés en el espacio de escala se encuentran al detectar extremos en la diferencia de gaussianas de dos imágenes de espacio de escala adyacentes.



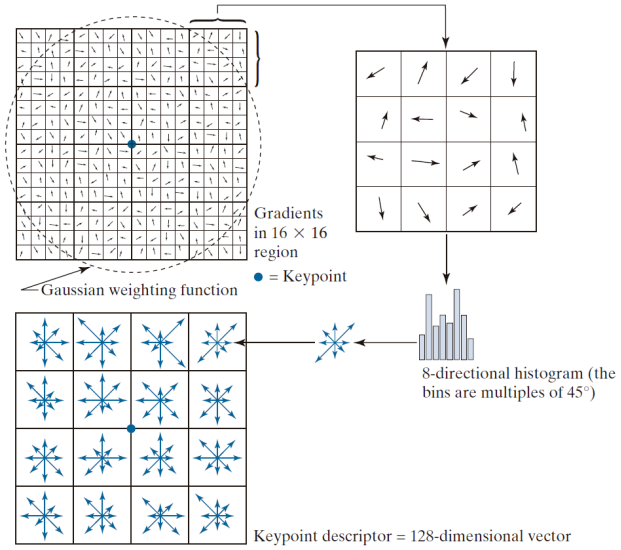
Orientación

- El siguiente paso es asignar una orientación consistente a cada punto de interés en función de las propiedades de la imagen local. Esto permite representar un punto de interés relativo a su orientación y así lograr invariancia a la rotación de la imagen.
- Se construye un histograma de intensidades con 36 *bins* para cubrir los 360°, a partir de las intensidades del gradiente de los puntos de interés.
- El *bin* más alto indica la orientación del punto de interés, y para aquellos *bins* en el rango del 80% del más alto, se crean nuevos puntos de interés con esa orientación.



El descriptor

- Esta etapa busca calcular un descriptor para una región local alrededor de cada punto de interés que sea muy distintivo, pero al mismo tiempo sea lo más invariable posible a los cambios de escala, orientación, iluminación y punto de vista de la imagen.
- La idea es usar estos descriptores para identificar coincidencias (similitudes) entre regiones locales entre dos o más imágenes.
- SIFT utiliza las gradientes de la imagen suavizada para encontrar los descriptores locales.



Ejemplo SIFT en Python

In [7]:
```
import matplotlib.pyplot as plt
```

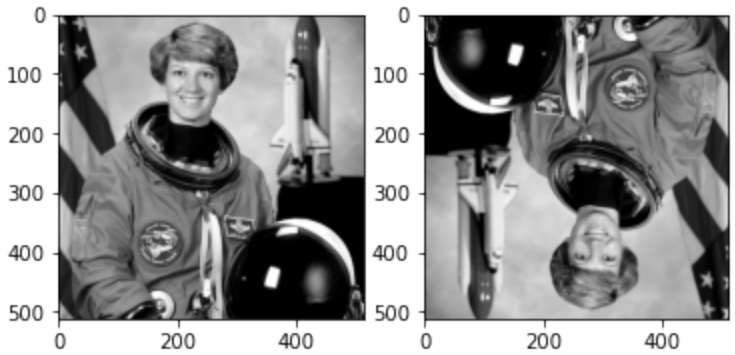
In [8]:
```
import skimage
# SIFT está disponible a partir de la versión 0.19.2
print(skimage.__version__)
# pip install -U scikit-image (si necesita actualizar)
```

```
0.19.2
```

In [9]:
```
from skimage import data
from skimage import transform
from skimage.color import rgb2gray
from skimage.feature import match_descriptors, plot_matches, SIFT
```

In [10]:
```
astronauta = rgb2gray(data.astronaut())
rotado = transform.rotate(astronauta, 180)
```

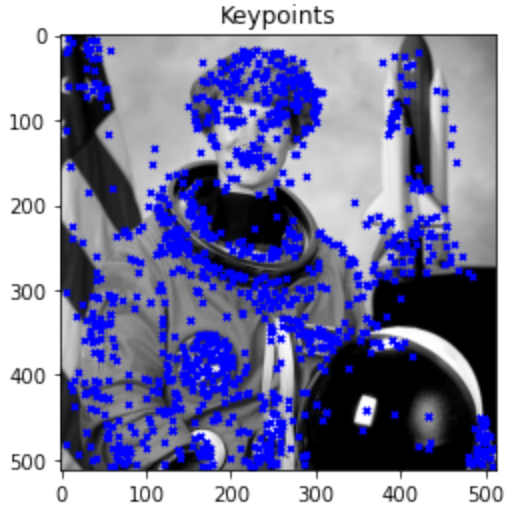
In [11]:
```
fig = plt.figure()
ax1 = plt.subplot(1, 2, 1)
ax2 = plt.subplot(1, 2, 2)
ax1.imshow(astronauta, cmap=plt.cm.gray)
ax2.imshow(rotado,  cmap=plt.cm.gray)
plt.show()
```



In [12]:
```
descriptor_extractor = SIFT()
```

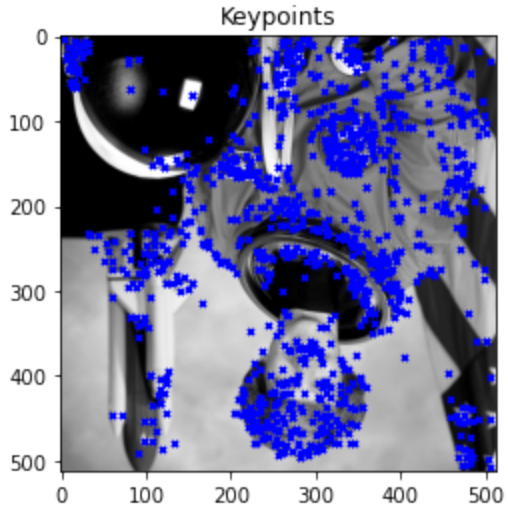
In [13]:
```
descriptor_extractor.detect_and_extract(astronauta)
keypoints1 = descriptor_extractor.keypoints
descriptors1 = descriptor_extractor.descriptors
```

In [14]:
```
plt.imshow(astronauta,  cmap=plt.cm.gray)
plt.scatter(keypoints1[:, 1], keypoints1[:, 0], s=10, c='b', marker="x")
plt.title("Keypoints")
plt.show()
```



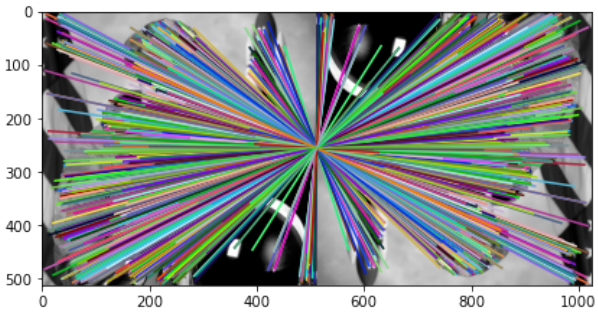
In [15]:
```
descriptor_extractor.detect_and_extract(rotado)
keypoints2 = descriptor_extractor.keypoints
descriptors2 = descriptor_extractor.descriptors
```

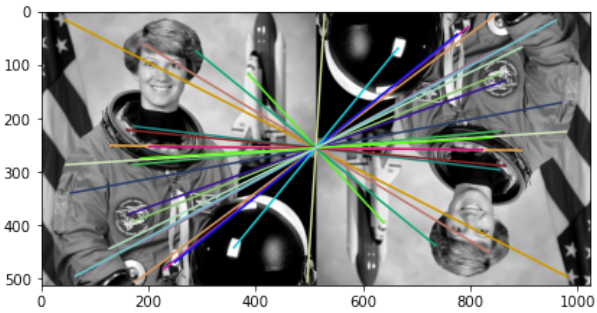
In [16]:
```
plt.imshow(rotado,  cmap=plt.cm.gray)
plt.scatter(keypoints2[:, 1], keypoints2[:, 0], s=10, c='b', marker="x")
plt.title("Keypoints")
plt.show()
```



In [17]:
```
matches = match_descriptors(descriptors1, descriptors2, max_ratio=0.6, cross_che
```

In [18]:
```
fig, ax = plt.subplots(nrows=1, ncols=2, figsize=(15, 15))
plot_matches(ax[0], astronauta, rotado, keypoints1, keypoints2, matches, only_ma
plot_matches(ax[1], astronauta, rotado, keypoints1, keypoints2, matches[::50], c
```





Template Matching

- Técnica para encontrar áreas dentro de una imagen que coinciden (*match*, son similares) a una imagen *template*.
- Se busca detectar el área de mayor coincidencia.

Pasos del Template Matching

- Para identificar el área coincidente, es necesario comparar la imagen *template* con la imagen *fuente*, deslizándola.
- Mover el *template* un pixel a la vez (de izquierda a derecha y de arriba a abajo).
- En cada posición, se utiliza una métrica de similaridad (e.g., coeficiente de correlación) para calcular *matching* (o cuán similar es el *template* a esa área en particular de la imagen *fuente*).
- Para cada posición (x, y) del *template* sobre la *fuente*, se almacena el resultado de la métrica en una matriz de resultados.
- Las posiciones más brillantes indican las mejores coincidencias.
- Se entrega un rectángulo formado por el punto más brillante como esquina, y un ancho y alto igual al del *template*.

In [19]:
```
from skimage import data
from skimage.feature import match_template
import numpy as np
```

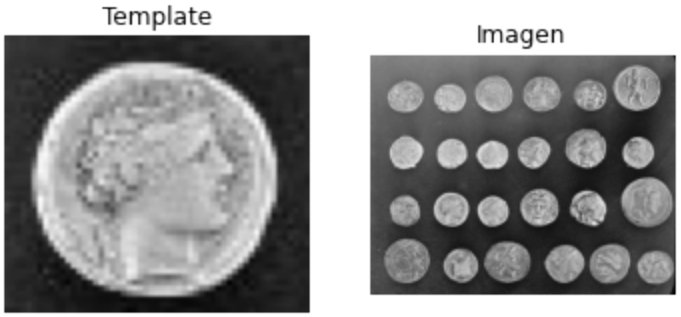
In [20]:
```
image = data.coins()
coin = image[170:220, 75:130]
```

In [21]:
```
fig = plt.figure()
ax1 = plt.subplot(1, 2, 1)
ax2 = plt.subplot(1, 2, 2)

ax1.imshow(coin, cmap=plt.cm.gray)
ax1.set_axis_off()
ax1.set_title('Template')

ax2.imshow(image, cmap=plt.cm.gray)
ax2.set_axis_off()
```

```
ax2.set_title('Imagen')
plt.show()
```



In [22]:

```
result = match_template(image, coin)
ij = np.unravel_index(np.argmax(result), result.shape)
x, y = ij[::-1]
```

In [23]:

```
def DisplayTemplateMatching(image, template):
    fig = plt.figure(figsize=(10, 5))
    ax1 = plt.subplot(1, 3, 1)
    ax2 = plt.subplot(1, 3, 2)
    ax3 = plt.subplot(1, 3, 3)

    ax1.imshow(template, cmap=plt.cm.gray)
    ax1.set_axis_off()
    ax1.set_title('Template')

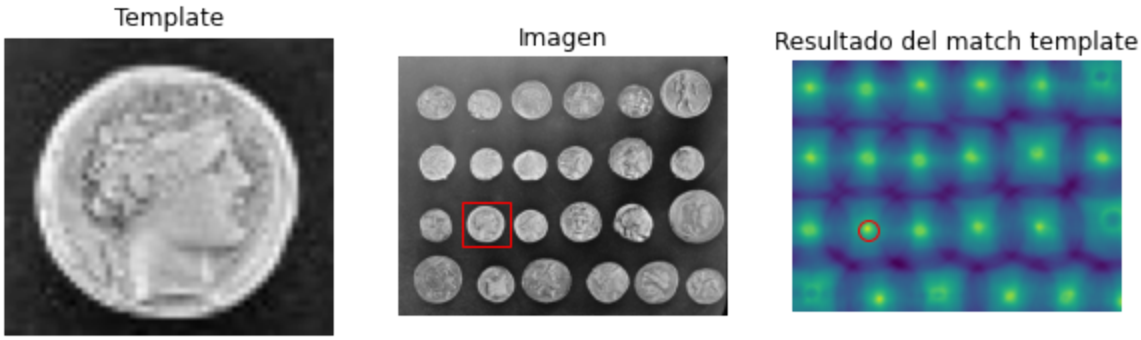
    ax2.imshow(image, cmap=plt.cm.gray)
    ax2.set_axis_off()
    ax2.set_title('Imagen')
    # highlight matched region
    hcoin, wcoin = template.shape
    rect = plt.Rectangle((x, y), wcoin, hcoin, edgecolor='r', facecolor='none')
    ax2.add_patch(rect)

    ax3.imshow(result)
    ax3.set_axis_off()
    ax3.set_title('Resultado del match template')
    # highlight matched region
    ax3.autoscale(False)
    ax3.plot(x, y, 'o', markeredgecolor='r', markerfacecolor='none', markersize=

    plt.show()
```

In [24]:

```
DisplayTemplateMatching(image, coin)
```



Histogram matching

- Manipula los píxeles de una imagen de entrada para que su histograma coincida con el histograma de la imagen de referencia. Si las imágenes tienen múltiples canales, el emparejamiento se realiza de forma independiente para cada canal, siempre que el número de canales sea igual en la imagen de entrada y la referencia.
- Se puede utilizar como una normalización ligera para el procesamiento de imágenes, especialmente en circunstancias en las que las imágenes se han tomado de diferentes fuentes o en diferentes condiciones (es decir, iluminación).

In [25]:
```
from skimage import exposure
from skimage.exposure import match_histograms
```

In [26]:
```
reference = data.coffee()
image = data.chelsea()
```

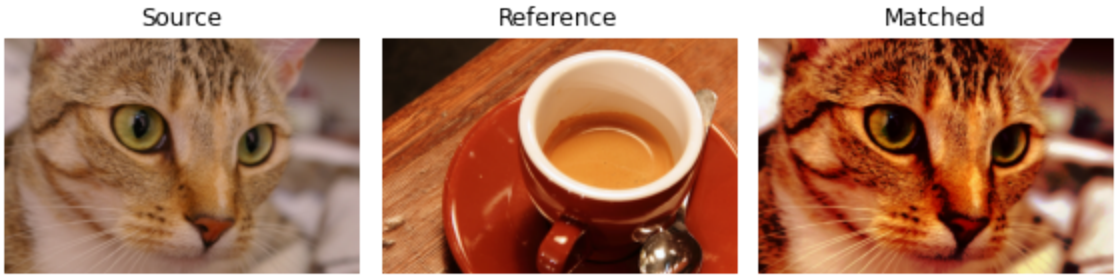
In [27]:
```
matched = match_histograms(image, reference, channel_axis=2)
```

In [28]:
```
fig, (ax1, ax2, ax3) = plt.subplots(nrows=1, ncols=3, figsize=(8, 3), sharex=Tru

for aa in (ax1, ax2, ax3):
    aa.set_axis_off()

ax1.imshow(image)
ax1.set_title('Source')
ax2.imshow(reference)
ax2.set_title('Reference')
ax3.imshow(matched)
ax3.set_title('Matched')

plt.tight_layout()
plt.show()
```



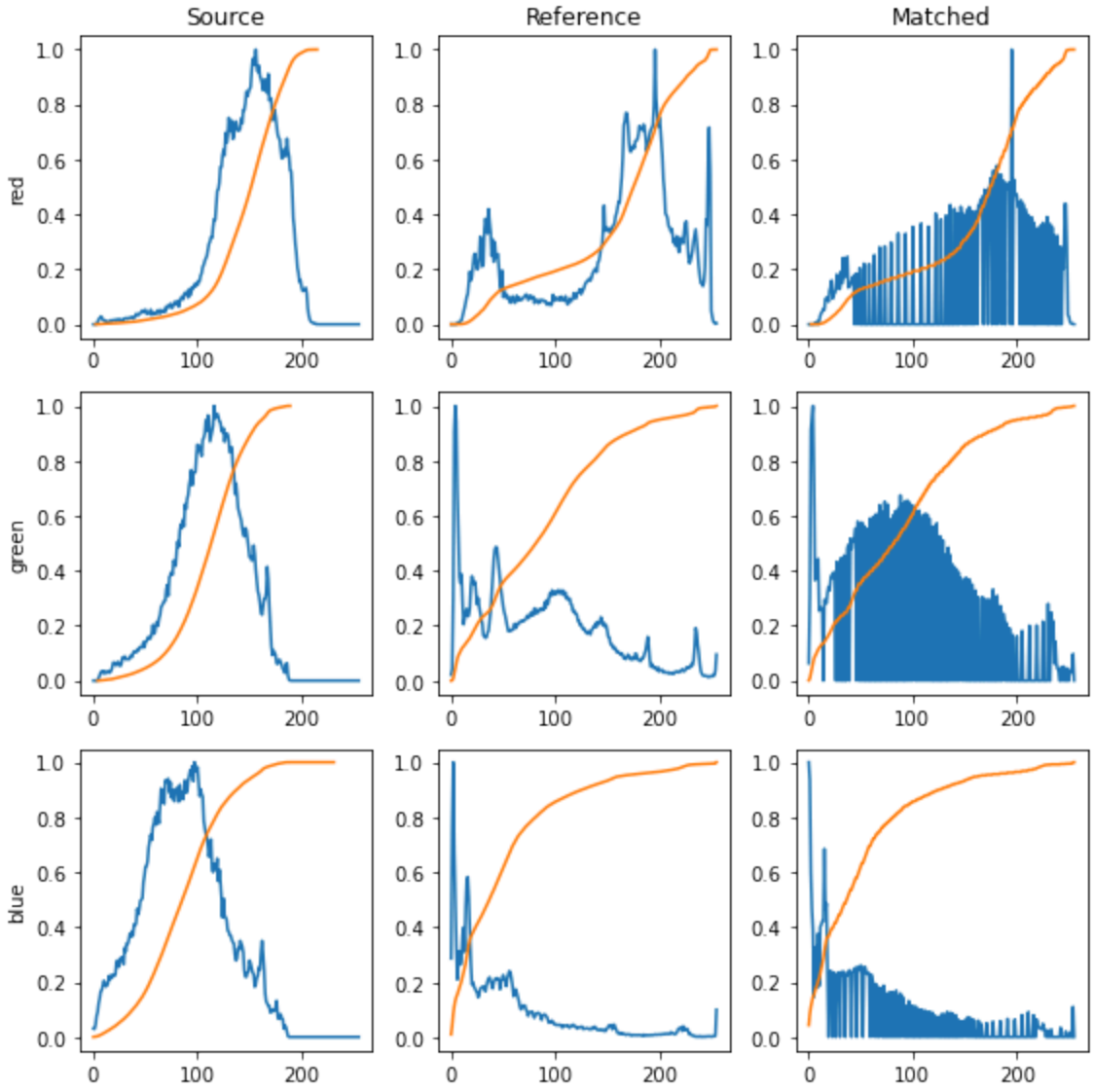
In [29]:

```
fig, axes = plt.subplots(nrows=3, ncols=3, figsize=(8, 8))

for i, img in enumerate((image, reference, matched)):
    for c, c_color in enumerate(('red', 'green', 'blue')):
        img_hist, bins = exposure.histogram(img[..., c], source_range='dtype')
        axes[c, i].plot(bins, img_hist / img_hist.max())
        img_cdf, bins = exposure.cumulative_distribution(img[..., c])
        axes[c, i].plot(bins, img_cdf)
        axes[c, 0].set_ylabel(c_color)

axes[0, 0].set_title('Source')
axes[0, 1].set_title('Reference')
axes[0, 2].set_title('Matched')

plt.tight_layout()
plt.show()
```


Source
Reference
Matched
red
green
blue
0.0
0.2
0.4
0.6
0.8
1.0
0
100
200